

Certificate specification

Repository source: `certificate/CERTIFICATE_SPECIFICATION.md`

Contents

Purpose and schema	1
Directory contents	1
Authoritative and predecessor JSON objects	1
NPZ proof objects	2
Exact/audit renderings	2
Final JSON structure	2
theorem	2
circumradius_partition	3
lower_branch	3
near_Jung_branch	3
pi_bounds	3
claim_boundary	3
NPZ field specification	4
Fan objects	4
Refined lower-cell objects	4
TSV and CSV columns	4
refined_lower_rows.tsv	4
fan_orbit_margin_audit.tsv	5
all_axis_pair_section_hulls.tsv	5
audit_constants.csv	5
Deterministic reconstruction	5
Versioning rule	6
Validation commands	6

Purpose and schema

The authoritative assembled certificate is:

```
certificate/certificates/universal_contact_bound_certificate.json
```

Its schema identifier is:

```
minvol.universal_contact_bound.v1
```

The certificate records the exact theorem coefficient, a gap-free circumradius partition, the complete lower-branch row table, the near-Jung fan and cap summaries, rational bounds for π , and an explicit claim boundary. The exact verifier rebuilds this JSON deterministically and requires byte-identical agreement with the committed file.

Directory contents

Authoritative and predecessor JSON objects

File	Role
<code>universal_contact_bound_certificate.json</code>	Final assembled theorem certificate

File	Role
<code>reference_handoff_certificate.json</code>	Immutable predecessor-row and reference-fan regression data used by the standalone verifier

The reference JSON is a narrow immutable input. It is not presented as a second standalone theorem certificate.

NPZ proof objects

File	Role
<code>reference_near_jung_fan.npz</code>	Immutable reference fan for nesting and multiplier regression
<code>universal_contact_fan.npz</code>	Adjusted 98,306-ray fan used by the final near-Jung branch
<code>refined_lower_cell_1.npz</code> through <code>refined_lower_cell_4.npz</code>	Four exact doubled-angular-grid cells replacing the old 03A1 interval

All NPZ files are opened with `allow_pickle=False`.

Exact/audit renderings

File	Role
<code>refined_lower_rows.tsv</code>	Human-readable exact lower replacement rows
<code>fan_orbit_margin_audit.tsv</code>	One representative margin row per stabilizer orbit
<code>all_axis_pair_section_hulls.tsv</code>	Exact vertices for all positive/negative axis section hulls
<code>audit_constants.csv</code>	Exact numerators, denominators, and decimal renderings used by numerical audits

The exact verifier regenerates these derived text artifacts and requires byte-identical agreement.

Final JSON structure

The top-level keys are:

```

schema
status
theorem
circumradius_partition
lower_branch
near_Jung_branch
pi_bounds
claim_boundary

```

theorem

Required fields:

Field	Meaning
<code>coefficient_of_pi</code>	Exact rational multiplying πd^3
<code>coefficient_decimal_lower</code>	Certified decimal lower rendering
<code>coefficient_decimal_upper</code>	Certified decimal upper rendering
<code>statement</code>	Plain-text theorem statement
<code>strict_gain_over_previous_lower</code>	Exact-decimal gain over the immediate predecessor certificate

For schema v1, `coefficient_of_pi` is 26221074253/200000000000.

circumradius_partition

This object records:

- the exact rational split;
- a decimal rendering of the split;
- the closed lower interval;
- the closed near-Jung interval; and
- a Boolean gap-free flag.

The common endpoint is included in both branches, which is harmless because both branch certificates apply there.

lower_branch

This object contains:

- the SHA-256 fingerprint of the reference handoff certificate;
- the predecessor row replaced by the four new cells;
- the refined angular/radial grid parameters;
- the complete ordered row list;
- the unique weakest row; and
- the exact weakest coefficient of π .

Each row records at least:

```
case
interval
s0
positive_steps
negative_steps
coefficient_of_pi
```

The four refined rows additionally record the angular and radial grids and their exact margins over the displayed theorem coefficient.

near_Jung_branch

This object records:

- reference and adjusted fan fingerprints;
- fan vertices, faces, edges, orbits, and minimum cone determinant;
- changed-vertex and active-bound censuses;
- the exact minimum fan margin and its index;
- determinant and total-deficiency bounds;
- the dominant-edge coordinate-width lemma data;
- the forced-core lower bound;
- three axis-specific cap-pair records;
- total cap contribution;
- cross-axis disjointness data;
- the near-Jung volume lower bound; and
- the strict handoff margin over the theorem upper enclosure.

The exact values can be very large rational numbers. Public prose should cite the JSON field and a certified decimal rendering rather than copying long fractions manually.

pi_bounds

The certificate stores rational lower and upper bounds for π . These are used only where a strict decimal comparison requires an exact enclosure. Floating-point values of π are not theorem authority.

claim_boundary

This object explicitly separates:

- statements certified inside the package; and
- statements not established by the package.

Release documents must remain consistent with this boundary.

NPZ field specification

Fan objects

Both fan NPZ files contain:

Key	Shape	Type	Meaning
Aint	(98306,)	int64	Radial integer numerators on the canonical fan rays
delta_num	(98306,)	int64	Dyadic S-procedure multiplier numerators
R_num	scalar	int64	Circumradius numerator
R_den	scalar	int64	Circumradius denominator
DBITS	scalar	int64	Dyadic denominator exponent for multipliers
volnum	scalar string	text	Exact fan-volume numerator
volden	scalar string	text	Exact fan-volume denominator

The exact verifier checks shapes, positivity, reference nesting, multiplier inheritance, orbit margins, fan volume, and declared fingerprints.

Refined lower-cell objects

Each refined cell contains:

Key	Type	Meaning
rlo_num, rlo_den	scalar int64	Left circumradius endpoint
rhi_num, rhi_den	scalar int64	Right circumradius endpoint
s0_num, s0_den	scalar int64	Tangency radius
coef_num, coef_den	scalar int64	Certified coefficient of π
w_num	one-dimensional int64 array	Positive-contact radial chain numerators
z_num	one-dimensional int64 array	Antipodal radial upper-chain numerators
N	scalar int64	Tangent-half-angle denominator
grid	scalar int64	Common radial denominator

The four `w_num` arrays have 12,740, 12,720, 12,700, and 12,680 entries. The four `z_num` arrays have 36,890, 36,931, 36,973, and 37,015 entries.

TSV and CSV columns

`refined_lower_rows.tsv`

```
case
left
right
s0
angle_N
radial_grid
```

```
positive_steps
negative_steps
coefficient_of_pi
margin_over_theorem
```

fan_orbit_margin_audit.tsv

```
representative
orbit_size
radial_numerator
delta_numerator
negative_count
active_bound
margin_decimal
```

all_axis_pair_section_hulls.tsv

```
axis
sign
index
u_num
u_den
v_num
v_den
```

audit_constants.csv

```
name
numerator
denominator
decimal
```

The decimal column is an audit rendering. The numerator and denominator are the exact record.

Deterministic reconstruction

The exact program follows this order:

1. verify immutable fingerprints;
2. load and semantically verify the four refined lower cells;
3. assemble the complete ordered lower row list and check gap-free coverage;
4. load reference and adjusted fans;
5. check nesting, inherited multipliers, ray/orbit margins, and fan volume;
6. reconstruct section hulls and exact areas;
7. verify determinant and coordinate-width bounds;
8. verify cap allocations and cross-axis disjointness;
9. assemble the near-Jung lower bound and handoff margin;
10. compare both branches, verify 03B02A/03B02B clear the theorem coefficient, and identify the near-Jung branch as the universal bottleneck;
11. write or compare derived TSV/CSV files; and
12. serialize the final JSON with sorted keys and two-space indentation.

The normal release path compares rather than rewrites committed proof objects. Deliberate regeneration should occur only when the mathematical certificate is being changed and reviewed.

Versioning rule

Any incompatible change to field meaning, array layout, arithmetic convention, or claim boundary requires a new schema identifier. Adding purely descriptive fields can remain within v1 only if old v1 consumers continue to interpret all existing fields identically.

Validation commands

From the repository root:

```
python3 certificate/certify_universal_bound.py
```

Run the reconstruction tests:

```
python3 -m unittest -v certificate/test_universal_bound.py
```

Run the adversarial mutations:

```
python3 certificate/test_adversarial_mutations.py
```

Verify the certificate checksum manifest:

```
cd certificate
```

```
shasum -a 256 -c SHA256SUMS.txt
```